

Atividade 4

Douglas Felipe

```
1  /*
2   * Tarefa 4 - Integracao Numerica Sequencial
3   *
4   * Metodo: Trapezio Composto
5   * Funcao:  $f(x) = x^2$ 
6   * Intervalo: [0, 2400000]
7   *
8   * O programa avalia diferentes numeros de subdivisoes N e mede
9   * o tempo de execucao da integracao sequencial.
10  *
11  * Para tornar a medicao mais estavel:
12  *   - sao feitos 3 aquecimentos;
13  *   - sao realizadas 20 medicoes;
14  *   - cada medicao executa o kernel 1000 vezes;
15  *   - o tempo do lote e normalizado para uma unica execucao;
16  *   - a mediana e utilizada como tempo representativo.
17  *
18  * A implementacao utiliza int64_t para evitar erros de ponto
19  * flutuante durante o calculo da integral.
20  */
21
22 #define _POSIX_C_SOURCE 199309L
23
24 #include <stdio.h>          /* printf, fprintf, perror */
25 #include <stdint.h>         /* int64_t, uint64_t */
26 #include <stdlib.h>         /* qsort, exit */
27 #include <inttypes.h>       /* PRIu64 */
28 #include <time.h>           /* clock_gettime, timespec */
29
30
31 /* Configuracao do experimento. */
32 #define NUM_AQUECIMENTOS    3
33 #define NUM_MEDICOES        20
34 #define REPETICOES_INTERNAS 1000
35
36 /* Intervalo de integracao. */
37 #define LIMITE_A INT64_C(0)
38 #define LIMITE_B INT64_C(2400000)
39
40 /*
41  * Valor analitico exato:
42  *
43  * Integral de  $x^2$  no intervalo [0, 2400000]
44  *
45  * =  $2400000^3 / 3$ 
46  * = 4608000000000000000
47  */
48 #define INTEGRAL_EXATA INT64_C(4608000000000000000)
49
50
51 /*
52  * Recebe o resultado calculado apos a regiao cronometrada.
53  *
54  * O uso de volatile aqui cria um efeito observavel e ajuda a impedir
55  * que o compilador considere as chamadas do kernel desnecessarias.
56  *
57  * O acumulador da integral NAO e volatile, pois isso alteraria
58  * artificialmente o desempenho do algoritmo.
59  */
60 static volatile int64_t destino_resultado = 0;
61
```

```

62
63 /*
64  * O kernel deve ser realmente executado em cada repeticao.
65  *
66  * noinline:
67  *   impede que a funcao seja incorporada diretamente ao chamador.
68  *
69  * noipa:
70  *   impede otimizacoes interprocedurais que poderiam fazer o GCC
71  *   perceber que chamadas repetidas com os mesmos argumentos
72  *   produzem sempre o mesmo resultado e reutilizar o valor.
73  */
74 #if defined(__GNUC__)
75 #define KERNEL __attribute__((noinline, noipa))
76 #else
77 #define KERNEL
78 #endif
79
80
81 /*
82  * Funcao utilizada na integracao:
83  *
84  *    $f(x) = x^2$ 
85  */
86 static inline int64_t funcao_integrando(int64_t x)
87 {
88     return x * x;
89 }
90
91
92 /*
93  * Calcula a integral utilizando o metodo composto do trapezio.
94  *
95  * Para N subdivisoes:
96  *
97  *    $h = (b - a) / N$ 
98  *
99  * Os pontos internos sao:
100  *
101  *    $x_i = a + i * h$ 
102  *
103  * A formulacao utilizada e:
104  *
105  *    $I = h * soma(f(x_i))$ 
106  *    $+ h * (f(a) + f(b)) / 2$ 
107  *
108  * com i variando de 1 ate N-1.
109  *
110  * O ponto  $x_i$  e calculado diretamente a partir de i, evitando
111  * dependencia entre  $x_i$  e  $x_{(i-1)}$ . Dessa forma, a dependencia
112  * sequencial relevante do loop fica concentrada no acumulador soma.
113  */
114 static KERNEL int64_t integral_trapezio(
115     int64_t a,
116     int64_t b,
117     int64_t n)
118 {
119     /* Largura de cada subdivisao do intervalo. */
120     const int64_t h = (b - a) / n;
121
122     /* Acumula os valores da funcao nos pontos internos. */
123     int64_t soma = 0;
124
125     for (int64_t i = 1; i < n; ++i) {
126
127         /*

```

```

128     * O ponto atual depende apenas de a, h e i.
129     * Nao depende do ponto calculado na iteracao anterior.
130     */
131     const int64_t x_i = a + i * h;
132
133     soma += funcao_integrando(x_i);
134 }
135
136 /* f(a) + f(b), que recebem peso 1 no metodo do trapezio. */
137 const int64_t extremos =
138     funcao_integrando(a) +
139     funcao_integrando(b);
140
141 /*
142     * Os pontos internos possuem peso 2 na formula tradicional.
143     *
144     * A expressao abaixo e uma forma equivalente reorganizada para
145     * evitar a multiplicacao intermediaria 2*soma, que poderia
146     * aproximar-se desnecessariamente do limite de int64_t.
147     */
148     return h * soma + (h * extremos) / 2;
149 }
150
151
152 /*
153     * Obtem um instante usando CLOCK_MONOTONIC.
154     *
155     * CLOCK_MONOTONIC representa tempo decorrido e nao sofre alteracoes
156     * caso o relógio do sistema seja ajustado.
157     */
158 static struct timespec obter_tempo(void)
159 {
160     struct timespec tempo;
161
162     if (clock_gettime(CLOCK_MONOTONIC, &tempo) != 0) {
163         perror("clock_gettime");
164         exit(EXIT_FAILURE);
165     }
166
167     return tempo;
168 }
169
170
171 /*
172     * Calcula a diferenca entre dois instantes em nanossegundos.
173     *
174     * timespec armazena separadamente:
175     *   - segundos;
176     *   - nanossegundos.
177     *
178     * O ajuste trata o caso em que tv_nsec do instante final e menor
179     * que tv_nsec do instante inicial devido a mudanca de segundo.
180     */
181 static uint64_t diferenca_ns(
182     struct timespec inicio,
183     struct timespec fim)
184 {
185     int64_t segundos =
186         (int64_t)fim.tv_sec - (int64_t)inicio.tv_sec;
187
188     int64_t nanos =
189         (int64_t)fim.tv_nsec - (int64_t)inicio.tv_nsec;
190
191     if (nanos < 0) {
192         --segundos;
193         nanos += INT64_C(1000000000);

```

```

194     }
195
196     return
197         (uint64_t)segundos * UINT64_C(1000000000) +
198         (uint64_t)nanos;
199 }
200
201
202 /*
203  * Funcao auxiliar utilizada pelo qsort para ordenar
204  * os tempos de execucao em ordem crescente.
205  */
206 static int comparar_uint64(const void *a, const void *b)
207 {
208     const uint64_t x = *(const uint64_t *)a;
209     const uint64_t y = *(const uint64_t *)b;
210
211     return (x > y) - (x < y);
212 }
213
214
215 /*
216  * Calcula a mediana das 20 medicoes.
217  *
218  * Como NUM_MEDICOES e par, a mediana corresponde a media
219  * dos dois valores centrais apos a ordenacao.
220  *
221  * A mediana e menos sensivel a medicoes isoladas afetadas
222  * por interrupcoes ou outros processos.
223  */
224 static double calcular_mediana(
225     uint64_t tempos[NUM_MEDICOES])
226 {
227     qsort(
228         tempos,
229         NUM_MEDICOES,
230         sizeof(uint64_t),
231         comparar_uint64
232     );
233
234     const size_t meio = NUM_MEDICOES / 2;
235
236     return
237         ((double)tempos[meio - 1] +
238         (double)tempos[meio]) / 2.0;
239 }
240
241
242 /*
243  * Calcula o resultado esperado pelo metodo do trapezio.
244  *
245  * Para  $f(x) = x^2$ , o erro do metodo composto e:
246  *
247  *  $erro = (b - a) * h^2 / 6$ 
248  *
249  * Como  $x^2$  e convexa, o metodo do trapezio superestima a integral:
250  *
251  *  $resultado\ esperado = integral\ exata + erro$ 
252  *
253  * Isso permite validar o resultado utilizando somente inteiros.
254  */
255 static int64_t resultado_esperado(
256     int64_t a,
257     int64_t b,
258     int64_t n)
259 {

```

```

260     const int64_t h = (b - a) / n;
261
262     const int64_t erro =
263         ((b - a) * h * h) / 6;
264
265     return INTEGRAL_EXATA + erro;
266 }
267
268
269 /*
270  * Executa o experimento para um valor de N.
271  *
272  * Etapas:
273  * 1. verificar N;
274  * 2. realizar aquecimentos;
275  * 3. realizar as medicoes;
276  * 4. validar o resultado;
277  * 5. calcular as metricas;
278  * 6. imprimir a linha correspondente.
279  */
280 static void executar_benchmark(int64_t n)
281 {
282     const int64_t a = LIMITE_A;
283     const int64_t b = LIMITE_B;
284
285     /*
286      * Como o experimento utiliza somente aritmetica inteira,
287      * h = (b-a)/N deve ser inteiro.
288      */
289     if (n <= 0 || (b - a) % n != 0) {
290         fprintf(
291             stderr,
292             "Erro: N=%" PRIu64 " nao divide o intervalo.\n",
293             n
294         );
295
296         exit(EXIT_FAILURE);
297     }
298
299     int64_t resultado = 0;
300     uint64_t tempos[NUM_MEDICOES];
301
302
303     /*
304      * Aquecimento.
305      *
306      * Sao executados tres lotes antes das medicoes.
307      * Os tempos dos aquecimentos nao sao armazenados.
308      */
309     for (int aquecimento = 0;
310          aquecimento < NUM_AQUECIMENTOS;
311          ++aquecimento) {
312
313         for (int r = 0; r < REPETICOES_INTERNAS; ++r) {
314             resultado = integral_trapezio(a, b, n);
315         }
316
317         destino_resultado = resultado;
318     }
319
320
321     /*
322      * Medicoes.
323      *
324      * Cada medicao cronometra 1000 execucoes do kernel.
325      * Isso aumenta a duracao da regio medida e reduz a influencia

```

```

326     * relativa do custo do temporizador.
327     */
328     for (int medicao = 0;
329         medicao < NUM_MEDICOES;
330         ++medicao) {
331
332         const struct timespec inicio = obter_tempo();
333
334         for (int r = 0; r < REPETICOES_INTERNAS; ++r) {
335             resultado = integral_trapezio(a, b, n);
336         }
337
338         const struct timespec fim = obter_tempo();
339
340         /* Uso observavel ocorre fora da regioao cronometrada. */
341         destino_resultado = resultado;
342
343         tempos[medicao] =
344             diferenca_ns(inicio, fim);
345     }
346
347
348     /*
349     * Validacao funcional.
350     *
351     * Os tempos somente sao aceitos se o resultado calculado
352     * for exatamente igual ao resultado esperado.
353     */
354     const int64_t esperado =
355         resultado_esperado(a, b, n);
356
357     if (resultado != esperado) {
358         fprintf(
359             stderr,
360             "Erro de validacao para N=%" PRId64 "\n",
361             n
362         );
363
364         exit(EXIT_FAILURE);
365     }
366
367
368     /* Erro em relacao ao valor analitico da integral. */
369     const int64_t erro_absoluto =
370         resultado - INTEGRAL_EXATA;
371
372
373     /*
374     * Tratamento das medicoes.
375     *
376     * Primeiro calcula-se a mediana do lote de 1000 execucoes.
377     */
378     const double mediana_lote_ns =
379         calcular_mediana(tempos);
380
381     /*
382     * Normalizacao para obter o tempo correspondente
383     * a uma unica execucao da integral.
384     */
385     const double mediana_execucao_ns =
386         mediana_lote_ns / REPETICOES_INTERNAS;
387
388     /* Tempo normalizado convertido para milissegundos. */
389     const double mediana_ms =
390         mediana_execucao_ns / 1000000.0;
391

```

```

392  /*
393  * Custo medio por trapezio.
394  *
395  * Essa metrica ajuda a observar o comportamento linear  $O(N)$ .
396  */
397  const double ns_por_trapezio =
398      mediana_execucao_ns / (double)n;
399
400
401  printf(
402      "%-12" PRId64
403      " %-22" PRId64
404      " %-16" PRId64
405      " %-14.6f"
406      " %-12.3f\n",
407      n,
408      resultado,
409      erro_absoluto,
410      mediana_ms,
411      ns_por_trapezio
412  );
413  }
414
415
416  int main(void)
417  {
418      /*
419      * Valores de N avaliados.
420      *
421      * Todos dividem exatamente 2400000, garantindo que
422      *  $h = (b-a)/N$  seja inteiro.
423      */
424      static const int64_t valores_n[] = {
425          INT64_C(24000),
426          INT64_C(120000),
427          INT64_C(240000),
428          INT64_C(400000),
429          INT64_C(800000),
430          INT64_C(1200000),
431          INT64_C(2400000)
432      };
433
434      const size_t quantidade =
435          sizeof(valores_n) / sizeof(valores_n[0]);
436
437
438      /* Informacoes gerais do experimento. */
439      printf("Integracao Sequencial - Metodo do Trapezio\n");
440      printf("f(x)          : x^2\n");
441      printf(
442          "Intervalo      : [%" PRId64 " , %" PRId64 "]\n",
443          LIMITE_A,
444          LIMITE_B
445      );
446      printf("Tipo numerico : int64_t\n");
447      printf("Relogio       : CLOCK_MONOTONIC\n");
448      printf("Aquecimentos  : %d\n", NUM_AQUECIMENTOS);
449      printf("Medicoes      : %d\n", NUM_MEDICOES);
450      printf(
451          "Repeticoes     : %d por medicao\n\n",
452          REPETICOES_INTERNAS
453      );
454
455
456      /* Cabecalho da tabela de resultados. */
457      printf(

```

```
458         "%-12s %-22s %-16s %-14s %-12s\n",
459         "N",
460         "Integral",
461         "ErroAbs",
462         "Mediana(ms)",
463         "ns/trap"
464     );
465
466     printf(
467         "-----\n"
468     );
469
470     /* Executa o experimento para cada valor de N. */
471     for (size_t i = 0; i < quantidade; ++i) {
472         executar_benchmark(valores_n[i]);
473     }
474
475
476     /* Mantem referencia ao destino observavel. */
477     (void)destino_resultado;
478
479     return EXIT_SUCCESS;
480 }
481 }
```